

Lesson 6 — Capstone: Breakout

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

A level is data

```
// 0 = nothing, 1 = brick, 2 = tough
const LEVEL_1 = [
  [1, 1, 1, 1, 1, 1, 1, 1],
  [1, 2, 1, 1, 1, 1, 2, 1],
  [0, 1, 1, 0, 0, 1, 1, 0]
];
```

You can **see** the level. Change it by typing digits. Add a level by adding an array.

Grid to objects

```
for (let y = 0; y < layout.length; y++) {
  for (let x = 0; x < layout[y].length; x++) {
    const kind = layout[y][x]; // ROW first!
    if (kind === 0) continue;
    bricks.push({
      x: LEFT + x * (BRICK_W + GAP),
      y: TOP + y * (BRICK_H + GAP),
      w: BRICK_W, h: BRICK_H,
      hitsLeft: kind, alive: true
    });
  }
}
```

`layout[y][x]` — the outer array holds **rows**.

`layout[x][y]` gives a level mirrored along the diagonal.

Which side was hit?

```
const overlapX = Math.min(b.x+b.w, k.x+k.w)
  - Math.max(b.x, k.x);
const overlapY = Math.min(b.y+b.h, k.y+k.h)
  - Math.max(b.y, k.y);

if (overlapY < overlapX) ball.speedY = -
ball.speedY;
else ball.speedX = -
ball.speedX;
```

The **smaller** overlap is the side it came in through. Always flipping `speedY` sends a side-clipping ball through the whole row.

One brick per frame

```
break; // after handling a hit
```

Without it a ball between two bricks destroys both and flips twice, so it carries straight on.

Removing things

```
// A: a flag, nothing is really removed
if (!brick.alive) continue;

// B: really remove, loop BACKWARDS
for (let i = a.length - 1; i >= 0; i--) {
  if (dead) a.splice(i, 1);
}
```

Decomposition

One job per function. If you cannot name it without "and", split it.

```
buildLevel · updatePaddle · updateBall ·
checkBrickCollisions · drawBricks · drawHud
```

Words

Data-driven — behaviour described by data, not code. **Decomposition** — one big job into small single-purpose jobs. **Refactor** — change the organisation, not the behaviour.

Section A — Recall

A1. Give two concrete advantages of storing a level as an array of digits rather than as sixty lines of code. [3 marks]

A2. In `layout[y][x]`, which index chooses the row? Explain why it is that one. [2 marks]

A3. What is the one-sentence test for whether a function is doing too much? [2 marks]

A4. A brick grid starts at $x = 32$, each brick is 68 wide with a 6-pixel gap. Where does the brick in column 4 start? Show your working. [3 marks]

A5. Give one reason to use an `alive` flag and one reason to actually remove an object from the array. [3 marks]

Section B — Predict the output

B1. How many bricks does this build, and what shape is the result? [4 marks]

```
const LEVEL = [  
  [1, 0, 1, 0],  
  [0, 1, 0, 1],  
  [2, 2, 2, 2]  
];
```

B2. Someone writes `layout[x][y]` instead of `layout[y][x]`, using the level from B1. Describe exactly what goes wrong. (Think carefully: the array is 3 rows of 4.) [4 marks]

B3. The ball is travelling right and slightly up. It clips the **left edge** of a brick. What does this code do, [4 marks] and what will the player see over the next few frames?

```
if (boxesOverlap(ballBox(), brick)) {  
  brick.alive = false;  
  ball.speedY = -ball.speedY;      // always vertical  
}
```

B4. The ball lands exactly in the gap between two bricks and overlaps both on the same frame. There is [3 marks] no `break`. What happens to the ball's direction, and why is that wrong?

Section C — Find and fix the bug

C1. Roughly half the bricks never disappear, even when clearly hit. You have seen this bug twice before [4 marks] in this course. Name it, explain it, and fix it.

```
for (let i = 0; i < bricks.length; i++) {  
  if (boxesOverlap(ballBox(), bricks[i])) {  
    bricks.splice(i, 1);  
  }  
}
```

C2. Destroyed bricks keep being drawn, but the ball passes straight through them. Where is the [3 marks] mistake?

C3. This `update` works, but it is bad code. Give two specific reasons, and say how you would [4 marks] restructure it.

```
function update(dt) {  
  // 140 lines: paddle input, ball movement, wall bounces, paddle collision,  
  // every brick collision, particles, shake, score, lives, level changes  
}
```

Section D — Build it

This is the capstone. You have 90 minutes. Start from your lesson 5 Pong, or from scratch if you prefer — most of it you have written before.

- ☐ **Checkpoint 1.** A paddle at the bottom you can steer, and a ball that bounces off the left, right and top walls. Falling off the bottom loses a life. (All of this is lessons 2–4.)
- ☐ **Checkpoint 2.** Write `LEVEL_1` as a 2-D array and `buildLevel()` to turn it into brick objects. Draw them. Change some digits and reload to prove it works.
- ☐ **Checkpoint 3.** Make the ball destroy bricks. Use the overlap comparison so it bounces off the correct side. Add the `break`.
- ☐ **Checkpoint 4.** Add score, lives and a HUD. Clearing every brick moves to the next level; running out of lives ends the game.
- ☐ **Checkpoint 5.** Add at least three levels, and make at least one of them interesting rather than just full. Add tough two-hit bricks that *look* different after the first hit.
- ☐ **Checkpoint 6.** Bring back the juice: sound, shake and particles. Then **change something to make the game yours** — a new rule, a new look, a new mechanic.

HOW THIS IS MARKED

Not on how many features you have. On four things: does it run without errors; is the level data

separate from the game logic; can you explain any function I point at; and did you change something to make it yours.

A slightly rougher game with an original idea in it beats a perfect copy of the example.

Section E — Design challenge

NO RIGHT ANSWER

E1. Design a level that is **fun** rather than merely full. Draw it as a grid of digits. Then write one sentence saying what makes it interesting to play.

Your level, as a grid of digits

E2. Your layouts are 8 columns wide. What would you have to change to support a 20-wide level? If the answer is "nothing", your code is good. If it is "several numbers", find them and say why they are there.

E3. Invent a level format that can also say "this brick is red", "this brick drops a power-up", "this brick cannot be broken". What does it look like? What did you give up to get it? (Hint: your current format is readable because each cell is one character.)

E4. The big one. You have built four games and all four had: a loop, a state machine, delta time, collision, and a score. **Design the part that is left over when you take Breakout out of Breakout.** What would a reusable game skeleton contain? What must always stay specific to one game?

Stretch goals

1. Power-ups dropped by bricks — a wider paddle, a second ball, a slower ball.
2. **A level editor.** Click a grid to toggle bricks, then print the array to the console so you can paste it back into your code. About 30 lines, and a genuinely useful tool.
3. Save the high score with `localStorage`. Wrap it in `try/catch` — it *throws* in a private window rather than returning null.
4. **Publish it.** See `code/PUBLISHING.md`. Then send someone the link.
5. **Make it yours.** Breakout with gravity. Breakout where the bricks move. Breakout for two players. This is the real one.